Java 骰子模擬器程式碼指導

https://gemini.google.com/app/5e3b7b43fb4be450

User prompt: import javax.swing.; public class Exercise1 { static JPanel pne=new JPanel(); static JButton btn=new JButton("擲骰子"); public static void main(String[] args) { JFrame frm=new JFrame("骰子模擬器"); pne.add(btn); frm.add(pne); frm.setSize(400, 320); frm.setVisible(true); frm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); } }幫我根據我目前的程式碼，告訴我哪裡有問題，以及提示我接下來可以怎麼寫*

Response: 芯妤，這是一個很好的起手式！你已經成功建立基本的視窗 (`JFrame`) 與按鈕 (`JButton`)，並且完成了標題、尺寸 (400x320) 以及關閉程式的設定。

針對你目前的程式碼，以下是需要微調的地方，以及接下來完成這項作業的實作提示：

目前程式碼需要調整的地方：

- **缺少視窗置中：** 規格要求視窗開啟時要置中。你可以在 `frm.setVisible(true);` 之前加入 `frm.setLocationRelativeTo(null);` 來達成這個效果。
- **版面配置 (Layout) 建議更改：** 你目前將按鈕放進 `JPanel`，預設會使用 `FlowLayout`（元件由左排到右）。因為規格明確要求「上方」顯示統計、「中央」顯示點數、「下方」放按鈕，建議可以直接使用 `BorderLayout` 進行排版，這會比單純用 `JPanel` 更好定位。

接下來可以怎麼寫（實作提示）：

- **1. 建立並配置 UI 元件 (JLabel)**
  - 你需要新增兩個 `JLabel`。一個用來顯示中央的點數，另一個用來顯示上方的統計資訊。
  - 使用 `BorderLayout` 把統計標籤加到 `BorderLayout.NORTH`，點數標籤加到 `BorderLayout.CENTER`，並把你的按鈕加到 `BorderLayout.SOUTH`。
  - 中央點數標籤的字體需要放大，可以使用 `標籤變數.setFont(new Font("Dialog", Font.PLAIN, 60))` 來設定 60pt 字體。
- **2. 宣告狀態變數**
  - 在 class 層級（和 `btn` 宣告在一起的地方）新增幾個 `static` 變數（例如 `int rollCount` 和 `int totalSum`），用來記錄「已擲次數 (N)」和「總和 (M)」。
- **3. 實作按鈕點擊事件 (ActionListener)**
  - 你需要幫按鈕加上監聽器：`btn.addActionListener(e -> { /* 邏輯寫這裡 */ });`。
  - **產生隨機數：** 在監聽器內，使用 `(int)(Math.random() * 6) + 1` 或是 `java.util.Random` 產生 1 到 6 的整數。
  - **設定顏色：** 寫一個簡單的 `if-else` 判斷，如果擲出 6，就使用 `標籤變數.setForeground(Color.GREEN)` 變綠色；如果是 1 變紅色 (`Color.RED`)；其他則設回黑色 (`Color.BLACK`)。
  - **更新畫面與計算：** 將隨機產生出來的數字更新到中央的 JLabel 上。接著將次數 N 加 1、總和 M 加上剛擲出的點數，計算出平均值後，更新上方的統計 JLabel。

- 小提醒：算平均值時要注意整數除法的問題，記得將數值轉型為 `double`，並搭配 `String.format("%.2f", average)` 來確保顯示格式為 X.XX。

你可以先試著把兩個 `JLabel` 放上視窗並調整好版面，接著再把按鈕的事件處理邏輯補上。實作過程中如果有遇到排版跑掉或是變數抓不到的問題，隨時可以再把程式碼貼上來討論！

User prompt: import javax.swing.; import java.util.Random; import java.awt.*; public class Exercise1 { static JPanel pne=new JPanel(); static JButton btn=new JButton("擲骰子"); static JLabel labNorth = new JLabel("已擲 0 次，總和 0，平均 0.00", JLabel.CENTER); static JLabel labCenter = new JLabel("?", JLabel.CENTER); static int count = 0; static int sum = 0; public static void main(String[] args) { JFrame frm=new JFrame("骰子模擬器"); pne.add(btn); frm.add(pne); frm.setSize(400, 320); frm.setVisible(true); frm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); } } 請問我目前寫到這裡有甚麼需要修改的地方嗎*

Response: 進度很棒！你已經成功宣告了需要的標籤 (`JLabel`) 以及用來計算的變數 (`count`, `sum`)。

目前程式碼在語法上沒有錯誤，但如果直接執行，**畫面上只會看到按鈕，不會出現那兩行文字標籤**。這是因為你雖然宣告了標籤，卻還沒有將它們「加入」視窗 (`JFrame`) 中，而且排版位置也需要明確指定。

以下是目前需要修改與補上的細節：

1. **將標籤加入視窗並指定位置：** `JFrame` 預設使用 `BorderLayout`。你需要明確告訴程式，`labNorth` 要在上方、`labCenter` 要在中央，而裝著按鈕的 `pne` 要在下方。
2. **設定中央標籤的字體大小：** 規格要求中央顯示點數的標籤字體必須為 60pt。
3. **視窗置中設定：** 加入 `frm.setLocationRelativeTo(null);`，讓程式開啟時能直接在螢幕正中央顯示。

你可以將 `main` 方法裡面的排版程式碼調整成這樣：

```
public static void main(String[] args)
{
    JFrame frm = new JFrame("骰子模擬器");

    // 1. 設定中央點數的字體大小 (60pt)
    labCenter.setFont(new Font("Dialog", Font.PLAIN, 60));

    // 2. 將按鈕放入 JPanel (你原本這行寫得很好)
    pne.add(btn);

    // 3. 將元件依序加入 JFrame，並明確指定 BorderLayout 的方位
    frm.add(labNorth, BorderLayout.NORTH);
    frm.add(labCenter, BorderLayout.CENTER);
    frm.add(pne, BorderLayout.SOUTH);

    frm.setSize(400, 320);
    frm.setLocationRelativeTo(null); // 4. 讓視窗開啟時置中

    // 建議將 setVisible 放在所有元件都設定完畢的最後一步
    frm.setVisible(true);
    frm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
```

把這段 UI 排版搞定後，你的外觀畫面就完全符合規格要求了！接下來的重頭戲就是幫 `btn` 加上 `ActionListener`，開始撰寫隨機數字、改變顏色跟計算平均的邏輯。先試試看更新這段程式碼，成功跑出畫面後，要接著挑戰按鈕邏輯時隨時可以再討論！

User prompt: 幫 btn 加上 ActionListener 為什麼要寫這個???

Response: 在 Java 的視窗程式設計中，按鈕本身只是畫面上一個「看起來可以按」的圖形，它完全不知道被點擊之後該做什麼反應。

`ActionListener`（動作監聽器）就像是派給這個按鈕的**專屬接線生**。當你用滑鼠點擊按鈕時，按鈕會發出一個動作事件（Action Event），而掛在按鈕上的 `ActionListener` 聽到這個信號後，就會立刻觸發並執行你寫在裡面的那段程式碼。

- **如果不加上它：** 你的「擲骰子」按鈕按下去就只會有視覺上的凹陷效果，程式不會有任何動作，標籤上的點數和統計數字也都不會改變。
- **加上它之後：** 你等於明確告訴程式：「只要有人按了這個按鈕，就立刻去執行『產生亂數、判斷顏色、計算平均』這些任務。」

簡單來說，按鈕只是一個觸發開關，`ActionListener` 才是真正賦予按鈕靈魂、讓它能與使用者互動並執行運算邏輯的關鍵機制！

User prompt: 那我是要用implements 將ActionListener裡面的功能寫出來嗎

Response: 你可以使用 `implements`，但**不是必須的**，而且針對這種單一按鈕的小程式，通常有更簡潔的寫法。

在 Java 中為按鈕加上 `ActionListener`，最常見的有以下兩種方式：

方法一：使用 Lambda 表達式（最推薦、最簡潔） 你完全不需要修改 `class Exercise1` 的第一行宣告。只要在 `main` 方法裡面，在設定視窗顯示之前，直接把事件綁定在 `btn` 上即可。這也是現代 Java 最常用的寫法：

```
btn.addActionListener(e -> {
    // 這裡就是你寫按鈕邏輯的地方
    // 例如：產生亂數、設定標籤顏色、更新次數與總和
    System.out.println("按鈕被點擊了！"); // 你可以先寫這行測試看看
});
```

(備註：這是 Java 8 之後支援的語法，它其實是簡化了 `new ActionListener() { public void actionPerformed(ActionEvent e) { ... } }` 這一長串程式碼。)

方法二：讓類別 `implements ActionListener`（你提到的傳統作法） 如果你使用這個方法，你需要做三件事：

1. 修改類別宣告：`public class Exercise1 implements java.awt.event.ActionListener`
2. 在 `class` 裡面額外新增一個方法：

   ```
   public void actionPerformed(java.awt.event.ActionEvent e) {
       // 邏輯寫在這裡
   }
   ```

3. 在 `main` 裡面設定綁定：`btn.addActionListener(new Exercise1());`

結論與建議： 因為你的 UI 元件目前都是 `static` 宣告，且都在 `main` 方法裡操作，**強烈建議直接使用「方法一」**。

你可以試著把方法一的程式碼加在 `main` 方法中 `frm.setVisible(true);` 的上方。確認點擊按鈕沒有報錯後，就可以開始在 `{ }` 裡面挑戰寫出產生 1~6 亂數的程式碼了！

User prompt: 請問方法一有其他種寫法嗎?我沒有很理解這種寫法

Response: 完全可以！你原本看到的 `e -> { ... }` 叫做「Lambda 表達式」，它是 Java 為了讓程式碼變短而發明的「簡寫」。初學階段看這種簡寫確實會覺得一頭霧水，因為它把背後的邏輯都隱藏起來了。

在學校的 Java 課程中，教授通常會先教最標準、最正統的寫法，叫做**匿名內部類別 (Anonymous Inner Class)**。這種寫法雖然字比較多，但結構非常清楚。

你可以改用下面這個標準寫法：

```
// 記得確認程式最上方有沒有 import 這兩個元件（如果沒有要自己補上）
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

// ... 在 main 方法裡面，設定 frm.setVisible(true); 之前加入這段：

btn.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        // 當按鈕被點擊時，大括號裡面的程式碼就會立刻被執行
        System.out.println("測試：按鈕成功被點擊！");

        // 接下來「產生亂數」、「設定顏色」、「計算平均」的程式碼，全部都要寫在這個區塊裡面
    }
});
```

這段程式碼在做什麼？

- **`new ActionListener() { ... }`**：這就像是我們「現場」建立了一個沒有名字的專屬接線生，並直接指派給這個按鈕。
- **`actionPerformed(ActionEvent e)`**：這是 Java 規定的標準動作指令。意思是「當按鈕被按下的動作發生時，請執行以下大括號內的內容」。

其實，前面那個簡短的 `e -> { ... }`，就只是 Java 幫你把 `new ActionListener() { public void actionPerformed(ActionEvent e)` 這一大串字全部自動省略掉而已！核心的運作原理是完全一樣的。

你可以先把這段標準寫法放進你的 `main` 方法裡，執行看看點擊按鈕時，底下主控台會不會印出測試文字。確認按鈕有反應後，我們就可以把產生 1 到 6 的隨機數邏輯寫進去了。想接著試試看隨機數怎麼寫嗎？

User prompt: import javax.swing.; import java.util.Random; import java.awt.*; import java.awt.event.ActionEvent; import java.awt.event.ActionListener; public class Exercise1 { static JPanel pne=new JPanel(); static JButton btn=new JButton("擲骰子"); static JLabel labNorth = new JLabel("已擲 0 次，總和 0，平均 0.00", JLabel.CENTER); static JLabel labCenter = new JLabel("?", JLabel.CENTER); static int count = 0; static int sum = 0; public static void main(String[] args) { JFrame frm = new JFrame("骰子模擬器"); // 1. 設定中央點數的字體大小 (60pt) labCenter.setFont(new Font("Dialog", Font.PLAIN, 60)); // 2. 將按鈕放入 JPanel (你原本這行寫得很好)*

pne.add(btn); // 3. 將元件依序加入 JFrame，並明確指定 BorderLayout 的方位 frm.add(labNorth, BorderLayout.NORTH); frm.add(labCenter, BorderLayout.CENTER); frm.add(pne, BorderLayout.SOUTH); frm.setSize(400, 320); frm.setLocationRelativeTo(null); // 4. 讓視窗開啟時置中 btn.addActionListener(new ActLis()); btn.addActionListener(new ActionListener()){ @Override public void actionPerformed(ActionEvent e) { // 當按鈕被點擊時，大括號裡面的程式碼就會立刻被執行 Random rand = new Random(); int num = rand.nextInt(6) + 1; if (num == 6) { labCenter.setForeground(Color.GREEN); } else if (num == 1) { labCenter.setForeground(Color.RED); } else { labCenter.setForeground(Color.BLACK); } } // 接下來「產生亂數」、「設定顏色」、「計算平均」的程式碼，全部都要寫在這個區塊裡面 } }; // 建議將 setVisible 放在所有元件都設定完畢的最後一步 frm.setVisible(true); frm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); } } 這是我目前寫的程式碼請你告訴我接下來要怎麼寫

Response: 你的亂數產生與顏色判斷邏輯寫得非常精確！但在順利執行前，需要先修正幾處語法排版錯誤，接著補上畫面文字更新與計算平均的邏輯就能大功告成了。

目前需要修正的語法小錯誤：

- **刪除多餘程式碼：** `btn.addActionListener(new ActLis());` 這一行需要刪除，因為程式裡並沒有建立 `ActLis` 這個類別，會導致無法編譯。
- **括號位置修正：** `btn.addActionListener(new ActionListener() ){` 多了一個 `)`，正確的寫法是 `new ActionListener() {`。

接下來要補上的 3 個步驟（寫在顏色判斷 `if-else` 的下方）：

1. **顯示骰子點數：** 將算出來的 `num` 顯示在中央的標籤上。
2. **更新統計變數：** 讓次數 (`count`) 加 1，並將擲出的數字累加到總和 (`sum`)。
3. **計算平均並更新文字：** 算出平均值（記得將數值轉型為 `double` 以保留小數點），然後用 `String.format` 更新上方的標籤。

以下是幫你修正語法並補上後續邏輯的完整 `main` 方法區塊：

```
public static void main(String[] args)
{
    JFrame frm = new JFrame("骰子模擬器");

    // 1. 設定中央點數的字體大小 (60pt)
    labCenter.setFont(new Font("Dialog", Font.PLAIN, 60));

    // 2. 將按鈕放入 JPanel
    pne.add(btn);

    // 3. 將元件依序加入 JFrame，並明確指定 BorderLayout 的方位
    frm.add(labNorth, BorderLayout.NORTH);
    frm.add(labCenter, BorderLayout.CENTER);
    frm.add(pne, BorderLayout.SOUTH);

    frm.setSize(400, 320);
    frm.setLocationRelativeTo(null); // 4. 讓視窗開啟時置中

    // 修正後的 ActionListener 寫法
    btn.addActionListener(new ActionListener() {
        @Override
        public void actionPerformed(ActionEvent e)
```

```
            {
                // 產生亂數
                Random rand = new Random();
                int num = rand.nextInt(6) + 1;

                // 判斷顏色
                if (num == 6) {
                    labCenter.setForeground(Color.GREEN);
                } else if (num == 1) {
                    labCenter.setForeground(Color.RED);
                } else {
                    labCenter.setForeground(Color.BLACK);
                }

                // --- 接下來是補上的邏輯 ---

                // 1. 更新中央標籤顯示的數字
                labCenter.setText(String.valueOf(num));

                // 2. 更新次數與總和
                count++;            // 相當於 count = count + 1;
                sum += num;         // 相當於 sum = sum + num;

                // 3. 計算平均值並更新上方標籤 (注意整數相除的問題，要先轉 double)
                double average = (double) sum / count;
                String info = String.format("已擲 %d 次，總和 %d，平均 %.2f", count, sum, average);
                labNorth.setText(info);
            }
        });

        // 建議將 setVisible 放在所有元件都設定完畢的最後一步
        frm.setVisible(true);
        frm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
```

將這段程式碼替換掉原本的 `main` 方法後，你的骰子模擬器應該就能完美運作了！試著執行看看數字、顏色和上方的統計資料有沒有如預期般變化。

User prompt: 為什麼我的視窗沒辦法跑出來

Response: 從你提供的終端機截圖來看，程式沒有跑出來是因為發生了編譯錯誤，終端機明確顯示了 `JFrame cannot be resolved to a type` 這個錯誤訊息。

這表示 Java 編譯器現在「不認識」`JFrame` 是什麼。通常發生這個問題，是因為在整理或複製程式碼的過程中，檔案最上方的 `import`（匯入）語法不小心被刪除或遺漏了。

解決方法： 請往上捲動你的程式碼，檢查 `Excercise1.java` 檔案的最上方（在 `public class ...` 那行之前），確認有沒有包含完整的 `import` 語句。請確保檔案頂端有這幾行：

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.Random;
```

把這幾行補回檔案最上方，重新存檔並執行，Java 就能重新找回 `JFrame` 以及其他顏色、事件相關的元件，你的骰子模擬器視窗就會順利跳出來了。

(另外一個小提醒：我注意到你 VS Code 左邊的檔名拼成 `Excercise1.java`，如果你裡面的類別名稱寫的是 `public class Exercise1`（少一個 c），Java 也會因為檔名與類別名稱不符而報錯。記得確保兩邊的拼法完全一致！)

先補上 `import` 試試看，如果還有紅字再跟我說！

User prompt: import javax.swing.; import java.awt.*; import java.awt.event.ActionEvent; import java.awt.event.ActionListener; import java.util.Random; class Excercise1 { public static void main(String[] args) { JFrame frm = new JFrame("骰子模擬器"); // 1. 設定中央點數的字體大小 (60pt) labCenter.setFont(new Font("Dialog", Font.PLAIN, 60)); // 2. 將按鈕放入 JPanel pne.add(btn); // 3. 將元件依序加入 JFrame，並明確指定 BorderLayout 的方位 frm.add(labNorth, BorderLayout.NORTH); frm.add(labCenter, BorderLayout.CENTER); frm.add(pne, BorderLayout.SOUTH); frm.setSize(400, 320); frm.setLocationRelativeTo(null); // 4. 讓視窗開啟時置中 // 修正後的 ActionListener 寫法 btn.addActionListener(new ActionListener() { @Override public void actionPerformed(ActionEvent e) { // 產生亂數 Random rand = new Random(); int num = rand.nextInt(6) + 1; // 判斷顏色 if (num == 6) { labCenter.setForeground(Color.GREEN); } else if (num == 1) { labCenter.setForeground(Color.RED); } else { labCenter.setForeground(Color.BLACK); } // --- 接下來是補上的邏輯 --- // 1. 更新中央標籤顯示的數字 labCenter.setText(String.valueOf(num)); // 2. 更新次數與總和 count++; // 相當於 count = count + 1; sum += num; // 相當於 sum = sum + num; // 3. 計算平均值並更新上方標籤 (注意整數相除的問題，要先轉 double) double average = (double) sum / count; String info = String.format("已擲 %d 次，總和 %d，平均 %.2f", count, sum, average); labNorth.setText(info); } }); // 建議將 setVisible 放在所有元件都設定完畢的最後一步 frm.setVisible(true); frm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); } }這是我修改完的版本*

Response: 你把 `import` 補齊了，而且類別名稱 `Excercise1` 也和你的檔名一致了！不過，你這次在貼上 `main` 方法時，**不小心把原本宣告在最上面的那些變數跟 UI 元件給覆蓋（刪除）掉了**。

因為程式碼碼裡面找不到 `labCenter`、`btn`、`count` 這些變數是在哪裡創造的，所以編譯器現在會出現一堆「找不到符號 (cannot find symbol)」的紅字錯誤。

只要把我們之前宣告的那 6 行變數加回 `class Excercise1` 和 `main` 方法之間，這支程式就完整了。以下是幫你拼合完整的最終版本：

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.Random;

public class Excercise1
{
    // --- 你不小心刪掉的這 6 行變數要記得補回來 ---
    static JPanel pne = new JPanel();
    static JButton btn = new JButton("擲骰子");
    static JLabel labNorth = new JLabel("已擲 0 次，總和 0，平均 0.00", JLabel.CENTER);
    static JLabel labCenter = new JLabel("?", JLabel.CENTER);
    static int count = 0;
    static int sum = 0;
    // ------------------------------------------------
```

```
    public static void main(String[] args)
    {
        JFrame frm = new JFrame("骰子模擬器");

        // 1. 設定中央點數的字體大小 (60pt)
        labCenter.setFont(new Font("Dialog", Font.PLAIN, 60));

        // 2. 將按鈕放入 JPanel
        pne.add(btn);

        // 3. 將元件依序加入 JFrame，並明確指定 BorderLayout 的方位
        frm.add(labNorth, BorderLayout.NORTH);
        frm.add(labCenter, BorderLayout.CENTER);
        frm.add(pne, BorderLayout.SOUTH);

        frm.setSize(400, 320);
        frm.setLocationRelativeTo(null); // 4. 讓視窗開啟時置中

        // 修正後的 ActionListener 寫法
        btn.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e)
            {
                // 產生亂數
                Random rand = new Random();
                int num = rand.nextInt(6) + 1;

                // 判斷顏色
                if (num == 6) {
                    labCenter.setForeground(Color.GREEN);
                } else if (num == 1) {
                    labCenter.setForeground(Color.RED);
                } else {
                    labCenter.setForeground(Color.BLACK);
                }

                // 1. 更新中央標籤顯示的數字
                labCenter.setText(String.valueOf(num));

                // 2. 更新次數與總和
                count++;            // 相當於 count = count + 1;
                sum += num;         // 相當於 sum = sum + num;

                // 3. 計算平均值並更新上方標籤 (注意整數相除的問題，要先轉 double)
                double average = (double) sum / count;
                String info = String.format("已擲 %d 次，總和 %d，平均 %.2f", count, sum, average);
                labNorth.setText(info);
            }
        });

        // 建議將 setVisible 放在所有元件都設定完畢的最後一步
        frm.setVisible(true);
        frm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

把這整段複製貼上蓋掉你現在的檔案，存檔後重新編譯執行，你的骰子模擬器就可以順利開始擲骰子並計算平均值了！
